

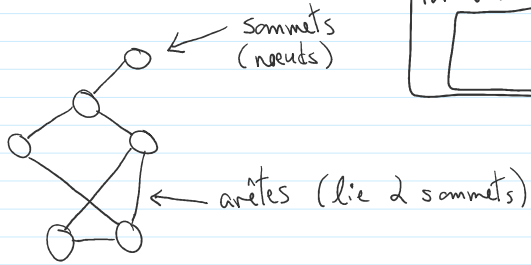
Graphes

```

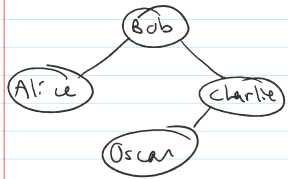
for i=1..n
  for j=1..n
    [ ]
  
```

```

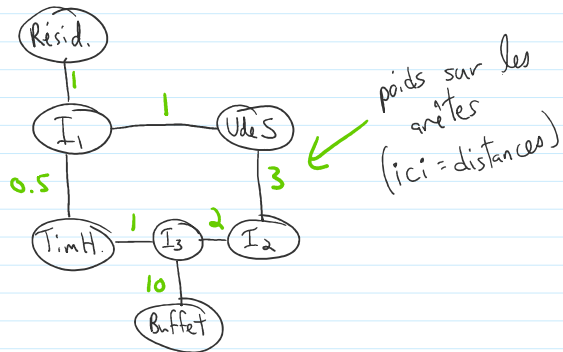
for i=1..n
  [ ] ← c
  
```



ex: réseau social : sommet = personne
arête = amitié



plan (carte)



Autres: • graphes d'interactions protéines
• www (serveurs trouteurs)
• ...

Graphe

sommets (vertex)



(/ /)

Graphes

On définit un graphe par $G=(V,E)$

Somme des sommets



↑ arêtes
(edge)

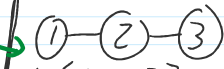
- V = ensemble quelconque
Souvent, $V = \{1, 2, \dots, n\}$

- E : paires non-ordonnées de V (sous-ens. de taille 2)

$$E \subseteq \{ \{u, v\} : u \in V, v \in V, u \neq v \}$$

- Un graphe peut être orienté. Dans ce cas,

$$E \subseteq \{ (u, v) : u \in V, v \in V, u \neq v \}$$



$$V = \{1, 2, 3\}$$

$$E = \{ \{1, 2\}, \{2, 3\} \}$$

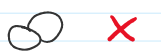
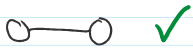


$$V = \{1, 2, 3\}$$

$$E = \{ (1, 2), (2, 3), (3, 2) \}$$

- Par défaut : non-orienté

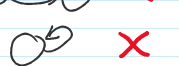
Non-orienté :



$$\textcircled{1} \quad V = \{1\}$$

$$E = \emptyset$$

Orienté :



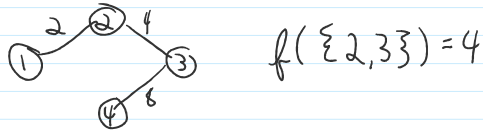
• Graphe valué

Un graphe $G=(V,E)$ est valué s'il est associé à une

Graphes pondérés

Un graphe $G=(V,E)$ est valué s'il est associé à une fct $f: E \rightarrow \mathbb{R}$ de poids.

On écrit alors $G=(V,E,f)$.



• $V(G), E(G)$

Soit G un graphe. On définit :

- $V(G)$: sommets de G $n = |V(G)|$
- $E(G)$: arêtes de G $m = |E(G)|$

• Pour $\{u,v\} \in E(G)$, on écrit uv .

• Si $uv \in E(G)$, on dit que u est voisin de v

↳ u et v sont adjacents

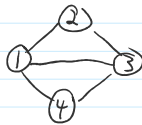
↳ u est incident à l'arête uv .

• On définit par $N_G(u)$ l'ensemble des voisins de u .

$$N_G(u) = \{v : v \in V(G) \text{ et } v \text{ est adjacent à } u\}$$

N : neighbor

(ou $N(u)$ si G est clair)



$$N(1) = \{2, 3, 4\}$$

• Si G est orienté :

$$N^+(u) = \{v : (u,v) \in E\}$$

$$N^-(u) = \{v : (v,u) \in E\}$$

• Degré

Pour $u \in V(G)$, le degré de u est son nb de voisins.

- Degré

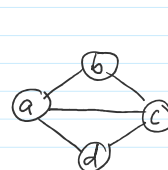
Pour $u \in V(G)$, le degré de u est son nb de voisins.

$$\deg(u) = |N(u)|$$

$$= |\{v : v \in V(G) \text{ et } uv \in E(G)\}|$$

Lemme: pour tout graphe $G=(V,E)$,

$$\sum_{u \in V(G)} \deg(u) = 2m = 2|E|$$



Preuve: quand on additionne tous les degrés, chaque arête est comptée exactement 2 fois: une fois pour chacun de ses sommets. ■

$$\begin{array}{l} \deg(a) = 3 \\ \deg(b) = 2 \\ \deg(c) = 2 \\ \deg(d) = 1 \\ \hline 10 = 2 \cdot 5 \\ = 2m \end{array}$$

- Graphe complet

Chaque arête possible est présente.

G est complet si: $E(G) = \{uv : u \in V(G), v \in V(G), u \neq v\}$

Q: combien d'arêtes y a-t-il dans un graphe complet?

$n=1$



$n=2$



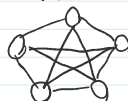
$n=3$



$n=4$



$n=5$



$m = 0$

1

3

6

10

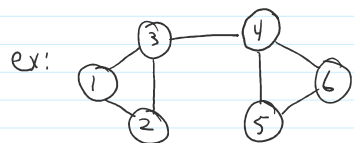
Rép: C'est le # de sous-ensembles de taille 2 d'un ens. de taille n :

$$\binom{n}{2} = \frac{n(n-1)}{2} \in \Theta(n^2)$$

$$\begin{array}{l} O(nm) = O(n \cdot n^2) \\ = O(n^3) \end{array}$$

Chemins, cycles, et connexité

Chemin : un chemin $P = (v_1, v_2, \dots, v_k)$ de sommets tous distincts t.g. $\forall i \in \{1, 2, \dots, k-1\}, v_i v_{i+1} \in E(G)$.



$\text{dist}(1, 5) = 3$

$P = (1, 2, 3, 4, 5, 6) = \text{chemin}$

$P = (2, 3, 4) = \text{chemin}$

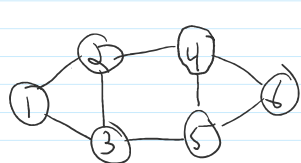
$P = (1, 3, 4, 6) = \text{chemin}$

$P = (1, 2, 5, 6) \neq \text{chemin}$

$P = (1, 3, 4, 3, 2) \neq \text{chemin}$

La longueur de P est $k-1$, i.e. le # d'arêtes à emprunter.

Cycle : un cycle $C = (v_1, v_2, \dots, v_k, v_1)$ est une séq. de sommets t.g. $(v_1, v_2, \dots, v_k)$ est un chemin et $v_k v_1 \in E(G)$.



$C = (1, 2, 4, 6, 5, 3, 1)$ $\text{long} = 6$

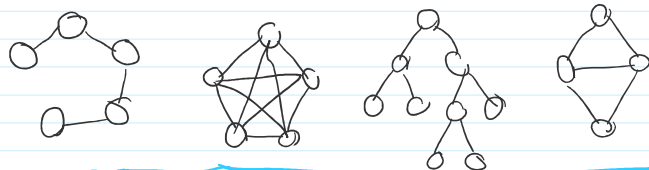
$C = (2, 4, 5, 3, 2)$ $\text{long} = 4$

La longueur de C est k , i.e. le # de sommets distincts (d'arêtes).

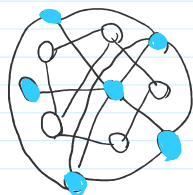
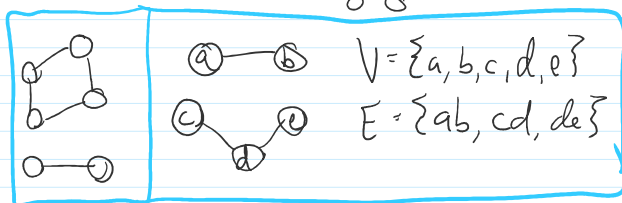
Connexe : Un graphe G est connexe si $\forall u, v \in V(G)$, il existe un chemin P de u à v .



ex:



Pas connexe:



Distance: soit $u, v \in V(G)$. La distance entre u et v est la longueur minimum d'un chemin de u à v .

Chemin le plus court

Entrée: graphe $G=(V,E)$ et deux sommets $u, v \in V(G)$.

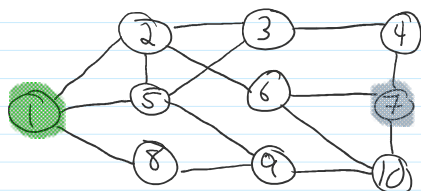
Sortie: un chemin P de u à v de longueur minimum.

Breadth-first search

Fouille en largeur

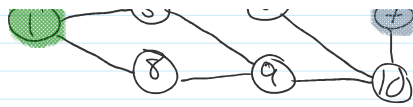
Idée:

- démarrer à u
- énumérer les sommets à $\text{dist}=1$
- énumérer les sommets à $\text{dist}=2$

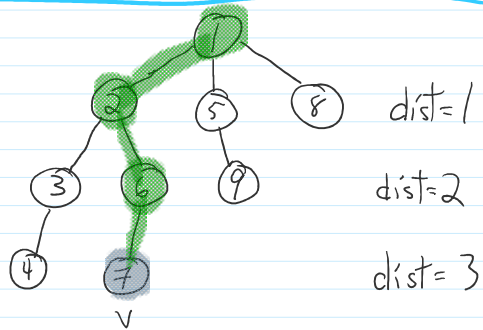


$u=1$

$v=7$



- énumérer les sommets à $\text{dist}=1$
- énumérer les sommets à $\text{dist}=2$
- ...
- ...
- ...
- arrêter quand on trouve v



$\text{dist}=1$

$\text{dist}=2$

$\text{dist}=3$

file: [1]
↓
1

Arbre de fille en largeur.

En pseudo-code:

- se rappeler des sommets visités.
↳ ensemble (hash table $\rightarrow$ insertion, requête $\in O(1)$)
- quand on ajoute un sommet à l'arbre
↳ file
- se rappeler du parent de chaque nœud énuméré
↳ dictionnaire: clé $\rightarrow$ valeur insertion, requête $\in O(1)$
sommets $\rightarrow$ parent

$\text{minChemin}(G=(V,E), u, v)$

visités = dict() // clé: sommet, val = parent

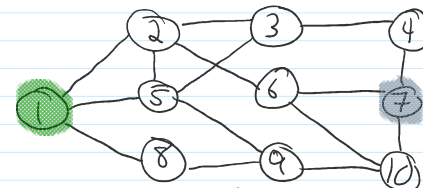
file = [u]

visités[u] = u

fini = false

file: [1, 2, 5, 8]

n Tours tant que $\neg \text{file.vide}()$ et fini = false



$u=1$

$v=7$

1 12 13 14 15 16 17 18 19 1n

n Tours
 $\deg(w)$
 $tours$
 $O(1)$

```

tini = false
tant que !file.vide() et fini = false
  w = file.pop() // obtient + enlève 1er elt
  pour chaque voisin z de w
    si z ≠ visités, clés()
      visités[z] = w
      file.append(z)
  si z == v
    fini = true
  x
  x
  x
  x
  x
  
```

// reconstruire un chemin

si $v \neq \text{visités}$
 return "pas de chemin"

$P = ()$

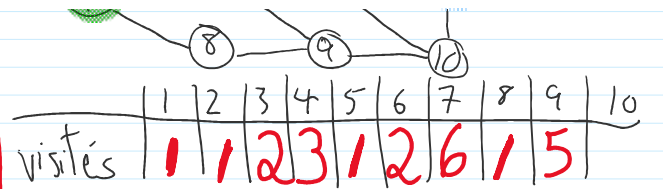
$cur = v$

tant que $cur \neq u$

$P.prepend(cur)$ // ajouter au début
 $cur = \text{visités}[cur]$

$P.prepend(u)$

return P



$file = [1, 2, 3, 4, 5, 6, 9, 4]$

$w = 1, 2, 3, 4, 5, 6$

$z = 3, 4, 5, \dots, 7$

Complexité: on fait $\leq n$ tours de la boucle principale
et on exécute un # c d'instructions avant
d'atteindre la boucle interne, $c \in O(1)$.

Pour chaque $w \in V(G)$, on exécute d instruction
dans la boucle interne $\deg(w)$ fois, $d \in O(1)$.

Le # d'instructions est au plus

$$n \cdot c + \sum_{w \in V(G)} \deg(w) \cdot d$$

(Lemme du dernier
cours)

$$= cn + d \cdot \sum_{w \in V(G)} \deg(w) = cn + d \cdot 2m$$

$$\in O(n + m).$$

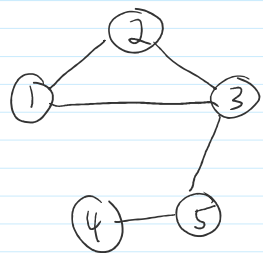
$$= O(|V(G)| + |E(G)|)$$

Sous-graphe induit

Pour $X \subseteq V(G)$, $G[X]$ dénote le sous-graphe induit par X .

On garde les sommets de X et leurs arêtes

$$G[X] = (X, \{uv : uv \in E(G) \text{ et } u \in X \text{ et } v \in X\})$$



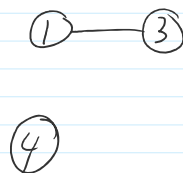
$$X = \{2, 3, 5\}$$

$$G[X] = G[\{2, 3, 5\}] =$$



$$X = \{1, 3, 4\}$$

$$G[X] =$$



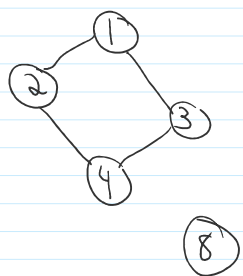
Composante connexe

On dit que $X \subseteq V(G)$ est une comp. connexe (CC) de G si

- $G[X]$ est connexe

- X est maximal par rapport à la connexité

↳ $\forall v \in V(G) \setminus X, G[X \cup \{v\}]$ n'est pas connexe



$$CC : \{1, 2, 3, 4\} \quad \{5, 6, 7\} \quad \{8\}$$

$$\text{Pas une CC: } \{1, 2, 3\}$$

Question: comment trouver les CC d'un graphe?

Avec la forêt en largeur.

fonction EnLargueur ($G=(V,E)$, u) // u = départ

visités = set()

visités.insert(u)

file = [u]

tant que $\neg$ file.vide()

w = file.pop()

 pour z voisin de w

 si $z \notin$ visités

 visités.insert(z)

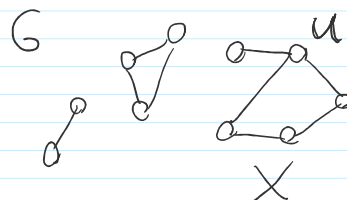
 file.append(z)

return visités

Complexité: $O(n+m)$

Plus précisément,
soit X la CC de G
qui contient u .

La complexité est
 $O(|X| + |E(G[X])|)$



Pour énumérer les CC

getCC($G=(V,E)$)

visités = set() // insert $O(1)$, et $\in O(1)$

lesCC = set() // val de retour

pour $w \in V(G)$

 si $w \notin$ visités

 CC-de- w = fonction EnLargueur(G, w)

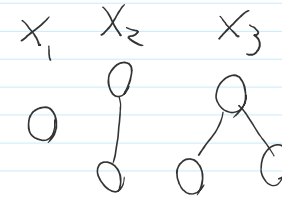
 lesCC.insert(CC-de- w)

 pour $z \in$ CC-de- w

```

    x ← CC-de-w
    pour z ∈ CC-de-w
    | visités, insert(z)
    x
return les CC

```



Complexité: soit $CC(G)$ l'ensemble des CC de G .
 Le temps est proportionnel à

$$\sum_{X \in CC(G)} (|X| + |E(G[X])|) = \sum_{X \in CC(G)} |X| + \sum_{X \in CC(G)} |E(G[X])|$$

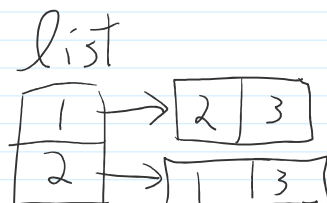
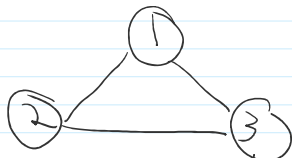
$$= n + m$$

$$\Rightarrow O(n+m)$$

Représentation d'un graphe

Liste d'adjacence: list, $\forall u \in V(G)$, list[u] = voisins de u

ex:

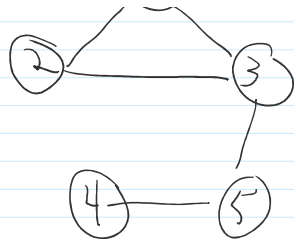


en C++

```

vector<vector<int>>
.. +   + .. + ..

```



1	→	[]
2	→	[1 3]
3	→	[5 2 1]
4	→	[5]
5	→	[3 4]

vector<vector<int>>
map<int, set<int>>

$O(n+m)$ mémoire

Matrice d'adjacence: Tableau $n \times n$ M où

$$M[u,v] = \begin{cases} 1 & \text{si } uv \in E(G) \\ 0 & \text{sinon} \end{cases}$$

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

$O(n^2)$ mémoire

<u>Complexité</u>	<u>Liste adj</u>	<u>Matrice adj</u>
Ajouter sommet (isolé)	$O(1)$	$O(n^2)$ (on doit tout recréer)
Ajouter une arête uv	$O(1)$	$O(1)$
Vérifier si $uv \in E$	$O(\deg(u))$	$O(1)$
Énumérer tous les voisins de u	$O(\deg(u))$	$O(n)$

$\Rightarrow$ À prouver: si G est connexe + acyclique,
 alors ① G est connexe (trivial)
 ② $m = n - 1$

On prouve $\forall$ arbre G , $m = n - 1$, par induction sur n .

Case de base: $n = 1$. Seul arbre: $\circ$ $m = 0 = n - 1$.

Induction. On suppose que si G est un arbre à $n-1$ sommets, alors G a $n-2$ arêtes.

Soit v une feuille de G . ($\deg(v)=1$). [v existe, exercice]

Soit $G-v$ le graphe duquel on retire v et son arête incidente.

$G-v$ est connexe car retirer v ne "coupe" pas de chemin.

$G-v$ est acyclique car retirer v ne peut pas créer de cycle.

Donc $G-v$ est un arbre. Par induction, le nb d'arêtes de $G-v$ est $|V(G-v)|-1 = n-1-1 = n-2$.

On remarque que G a une arête de plus que $G-v$. Donc $m = |E(G)| = n-2+1 = n-1$.

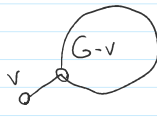
← si G est connexe et $m=n-1$, alors G est un arbre, i.e.
① G est connexe (trivial)
② G est acyclique

Exercice. Faisable par induction.

Si $m=n-1$, alors G a un sommet v de degré 1.

(car sinon, $\sum_{v \in V(G)} \deg(v) \geq \sum_{v \in V(G)} 2 = 2n$

$2m \geq 2n$
 $m \geq n$, contradiction.)

 par induction,
 $G-v$ acyclique,
et G aussi acyclique.